



Aror University of Art, Architecture, Design & Heritage Sukkur.

Department of Artificial Intelligence

Data Structures (Fall-2026)

LAB No. 2

Objective of Lab No. 1:

After performing lab1, students will be able to:

- Understand the basic concepts of Python data structures.
- Use Python lists, tuples, sets, and dictionaries for data organization.
- Implement basic operations associated with Abstract Data Types (ADTs).
- Understand the relationship between input size and algorithm performance.
- Analyze simple algorithms using Big-O notation.
- Compare the time complexity of common Python operations and algorithms.

Lab Tasks:

Task 01 — $O(1)$: Accessing an Array Element.

Task: Create a Python program that stores the marks of 10 students in a list. Access and display the marks of a student using its index. Measure and display the execution time and identify its time complexity.

Solution:

```
import time
```

```
marks = [75, 82, 91, 68, 88, 79, 95, 84, 73, 90]
```

```
# Measure execution time
```

```
start_time = time.perf_counter()
```

```
student_mark = marks[4]
```

```
end_time = time.perf_counter()
```

```
execution_time = end_time - start_time
```

```
print("Student Mark:", student_mark)
```

```
print("Execution Time:", execution_time, "seconds")
```

```
print("Time Complexity:  $O(1)$ ")
```



Aror University of Art, Architecture, Design & Heritage Sukkur.

Task 02 - $O(n)$: Finding the Sum of Elements

Task:

Create a Python function that calculates the sum of all numbers in a list. Test the function using different input sizes and measure the execution time.

Solution:

```
import time

def calculate_sum(data):
    total = 0

    for number in data:
        total += number

    return total

input_sizes = [100, 500, 1000, 2000]

print("Execution Time Comparison")
print("-" * 50)

for n in input_sizes:

    data = list(range(n))

    start_time = time.perf_counter()

    total = calculate_sum(data)

    end_time = time.perf_counter()

    execution_time = end_time - start_time

    print("Input Size:", n)
    print("Sum:", total)
    print("Execution Time:", execution_time, "seconds")
    print("Time Complexity:  $O(n)$ ")
    print("-" * 50)
```



Aror University of Art, Architecture, Design & Heritage Sukkur.

Task 03 — $O(n^2)$: Comparing Every Pair

Task:

Create a Python program that compares every element of a list with every other element. Count and display the total number of comparisons. Identify the time complexity.

Solution:

```
import time
```

```
def compare_pairs(data):
```

```
    comparisons = 0
```

```
    for i in data:
```

```
        for j in data:
```

```
            comparisons += 1
```

```
    return comparisons
```

```
data = [1, 2, 3, 4, 5]
```

```
start_time = time.perf_counter()
```

```
total_comparisons = compare_pairs(data)
```

```
end_time = time.perf_counter()
```

```
execution_time = end_time - start_time
```

```
print("Total Comparisons:", total_comparisons)
```

```
print("Execution Time:", execution_time, "seconds")
```

```
print("Time Complexity:  $O(n^2)$ ")
```



Aror University of Art, Architecture, Design & Heritage Sukkur.

Exercise: Implement the following tasks, find out the Time Complexity of each using `perf.counter()` method and classify the time complexity of each among $O(1)$, $O(n)$ and $O(n^2)$:

Task-1: Create a dictionary containing information about a student. Search for the student's semester using its key. Measure the execution time and identify the time complexity.

Task-2: Create a Python function that finds the highest mark from a list of student marks. Measure the execution time and identify the time complexity.

Task-3: Create a Python program using nested loops to generate multiplication tables from 1 to n . Measure the execution time for different values of n and identify the time complexity.

Task-4: Create a Python function that calculates the sum of all numbers in a list. Test the function using different input sizes and measure the execution time.

Task-5: Create a Python program that checks whether a list contains duplicate values by comparing each element with every other element. Measure the execution time and identify the time complexity.

Submission Requirements:

- Save the Python program/notebook using the format: `SAPID_YourName_Lab01`.
- Write your SAP ID and Name at the beginning of the notebook.
- Run all programs and include the output for each task.
- Add brief comments to explain important parts of the code.
- Submit the completed lab on the google classroom.